LAB ASSIGNMENT 16.1

Task 1 – Student Information System Schema

Task:

Design a database schema for a **Student Information System** and generate queries using AI.

Instructions:

- Tables: Students, Courses, Enrollments.
- Define primary keys, foreign keys, and relationships.
- Generate queries:
  - Insert a new student record.
  - Fetch all courses enrolled by a student.
  - Count number of students in each course.

Expected Output:

- SQL CREATE TABLE statements and queries for student–course relationships.

Code:

week 16 > student_info.sql

```sql
-- --create tables named as students and courses and enroll
CREATE TABLE IF NOT EXISTS students (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT NOT NULL,
    grade VARCHAR(10) NOT NULL
);
CREATE TABLE IF NOT EXISTS courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL
);
CREATE TABLE IF NOT EXISTS enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

```sql
--insert sample data into students table
INSERT INTO students (id, name, age, grade) VALUES
(1, 'Alice', 20, 'A'),
(2, 'Bob', 22, 'B'),
(3, 'Charlie', 19, 'A'),
(4, 'David', 21, 'C'),
(5, 'Eve', 20, 'B');
--insert sample data into courses table
INSERT INTO courses (course_id, course_name) VALUES
(1, 'Mathematics'),
(2, 'Physics'),
(3, 'Chemistry');
--insert sample data into enrollments table
INSERT INTO enrollments (enrollment_id, student_id, course_id) VALUES
(1, 1, 1),
(2, 1, 2),
(3, 2, 1),
(4, 3, 3),
(5, 4, 2);
```

```sql
--FETCH ALL COURSES ENROLLED BY STUDENTS
SELECT s.name AS student_name, c.course_name
FROM students s
JOIN enrollments e ON s.id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
--COUNT NO OF STUDENTS IN EACH COURSE
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

Output:

SQL ▾ 1 / 1

student_name	course_name
Alice	Mathematics
Alice	Physics
Bob	Mathematics
Charlie	Chemistry
David	Physics

SQL ▾ < 1 / 1 >

course_name	student_count
Chemistry	1
Mathematics	2
Physics	2

Task 2 – Hospital Management Database

Task:

Create schema and queries for a **Hospital Management System**.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
  - List all appointments for a specific doctor.
  - Retrieve patient history by patient ID.
  - Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

Code:

week 16 > hospital.sql

```sql
--create tables named as doctors and patients and appointments a
CREATE TABLE doc (
    doctor_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    specialization VARCHAR(50) NOT NULL
);
CREATE TABLE pat (
    patient_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT NOT NULL
);
CREATE TABLE app (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
);
```

```sql
-- insert sample data into doctors table
INSERT INTO doc (doctor_id, name, specialization) VALUES
(1, 'Dr. Smith', 'Cardiology'),
(2, 'Dr. Johnson', 'Neurology'),
(3, 'Dr. Williams', 'Pediatrics');
-- insert sample data into patients table
INSERT INTO pat (patient_id, name, age) VALUES
(1, 'Alice', 30),
(2, 'Bob', 25),
(3, 'Charlie', 35);
-- insert sample data into appointments table
INSERT INTO app (appointment_id, doctor_id, patient_id, appointment_date)
(1, 1, 1, '2024-07-01'),
(2, 1, 2, '2024-07-02'),
(3, 2, 3, '2024-07-03');
```

```sql
-- list all appointments for a doctor
SELECT d.name AS doctor_name, p.name AS patient_name, a.appointment_date
FROM doc d
JOIN app a ON d.doctor_id = a.doctor_id
JOIN pat p ON a.patient_id = p.patient_id
WHERE d.doctor_id = 1;
-- count no of patients for each doctor
SELECT d.name AS doctor_name, COUNT(a.patient_id) AS patient_count
FROM doc d
LEFT JOIN app a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.name;
```

SQL ▾ ‹ 1 / 1 › 1 - 2 of 2

doctor_name	patient_name	appointment_date
Dr. Smith	Alice	2024-07-01
Dr. Smith	Bob	2024-07-02

SQL ▾ ‹ 1 / 1 › 1 - 3 of 3

doctor_name	patient_count
Dr. Smith	2
Dr. Johnson	1
Dr. Williams	0

Task 3 – Library Database

Task:

Design schema for a **Library Management System**.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
  - Retrieve all books currently issued.
  - Find overdue books (loan date > 30 days).
  - Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Code:

week 16 > Library.sql

```sql
--create tables named as books and members and loans and retri
CREATE TABLE books (
    book_id INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    author VARCHAR(50) NOT NULL
);
CREATE TABLE members (
    member_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL
);
CREATE TABLE loans (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE NOT NULL,
    FOREIGN KEY (book_id) REFERENCES books(book_id),
    FOREIGN KEY (member_id) REFERENCES members(member_id)
);
```

```sql
-- insert sample data into books table
INSERT INTO books (book_id, title, author) VALUES
(1, 'The Great Gatsby', 'F. Scott Fitzgerald'),
(2, 'To Kill a Mockingbird', 'Harper Lee'),
(3, '1984', 'George Orwell');
```

```sql
-- insert sample data into members table
INSERT INTO members (member_id, name, email) VALUES
(1, 'Alice', 'alice@example.com'),
(2, 'Bob', 'bob@example.com'),
(3, 'Charlie', 'charlie@example.com');
-- insert sample data into loans table
INSERT INTO loans (loan_id, book_id, member_id, loan_date) VALUES
(1, 1, 1, '2024-06-01'),
(2, 2, 2, '2024-06-15'),
(3, 3, 3, '2024-05-20');
-- retrieve all books
SELECT * FROM books;
-- find overdue books (loan date > 30 days)
SELECT b.title, m.name, l.loan_date
FROM loans l
JOIN books b ON l.book_id = b.book_id
JOIN members m ON l.member_id = m.member_id
WHERE l.loan_date < DATE_SUB(CURDATE(), INTERVAL 30 DAY);
-- books per member
SELECT m.name, COUNT(l.loan_id) AS book_count
FROM members m
LEFT JOIN loans l ON m.member_id = l.member_id
GROUP BY m.member_id, m.name;
```

Output :

SQL ▾ < 1 / 1 > 1 - 3 of 3

book_id	title	author
1	The Great Gatsby	F. Scott Fitzgerald
2	To Kill a Mockingbird	Harper Lee
3	1984	George Orwell

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an **E-commerce platform**.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
  - Retrieve all orders by a user.
  - Find the most purchased product.
  - Calculate total revenue in a given month.
- AI should also suggest **normalization improvements** and **query optimization**.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

week 16 > ecom.sql

```sql
--create tables named as users and products and orders and ord
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL
);
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    price DECIMAL(10, 2) NOT NULL
);
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    order_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
CREATE TABLE orderDetails (
    order_detail_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

```sql
-- insert sample data into users table
INSERT INTO users (user_id, name, email) VALUES
(1, 'Alice', 'alice@example.com'),
(2, 'Bob', 'bob@example.com'),
(3, 'Charlie', 'charlie@example.com');
-- insert sample data into products table
INSERT INTO products (product_id, name, price) VALUES
(1, 'Laptop', 999.99),
(2, 'Smartphone', 499.99),
(3, 'Headphones', 199.99);
-- insert sample data into orders table
INSERT INTO orders (order_id, user_id, order_date) VALUES
(1, 1, '2024-07-01'),
(2, 2, '2024-07-02'),
(3, 3, '2024-07-03');
-- insert sample data into orderDetails table
INSERT INTO orderDetails (order_detail_id, order_id, product_id, quantity) VALUES
(1, 1, 1, 1),
(2, 1, 3, 2),
(3, 2, 2, 1),
(4, 3, 1, 1),
(5, 3, 2, 2);
```

```sql
-- retrive orders from user
SELECT u.name AS user_name, p.name AS product_name, od.quantity, o.order_date
FROM users u
JOIN orders o ON u.user_id = o.user_id
JOIN orderDetails od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id;
-- most purchased products
SELECT p.name AS product_name, SUM(od.quantity) AS total_quantity
FROM products p
JOIN orderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id, p.name
ORDER BY total_quantity DESC;
-- total revenue in a month
SELECT SUM(p.price * od.quantity) AS total_revenue
FROM orders o
JOIN orderDetails od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id
WHERE MONTH(o.order_date) = 7 AND YEAR(o.order_date) = 2024;
```

SQL ▾ < 1 / 1 > 1 - 1 of 1

order_id	user_id	order_date
1	1	2024-07-01

SQL ▾ < 1 / 1 > 1 - 1 of 1

product_id	total_sold
2	3

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

• Tables: Students, Subjects, Exams, Results.

• Define primary keys, foreign keys, and referential integrity constraints.

• Generate queries:

o Insert examination results for a student.

o Retrieve subject-wise marks for a student.

o Calculate average marks for each subject.

Expected Output:

• Normalized schema and SQL queries involving aggregation and joins.

week 16 > university.sql

```sql
--create tables named as students and subjects and exams and results a
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT NOT NULL
);
CREATE TABLE subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(50) NOT NULL
);
CREATE TABLE exams (
    exam_id INT PRIMARY KEY,
    subject_id INT,
    exam_date DATE NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
);
```

```sql
CREATE TABLE results (
    result_id INT PRIMARY KEY,
    student_id INT,
    exam_id INT,
    score FLOAT NOT NULL,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (exam_id) REFERENCES exams(exam_id)
);
```

```sql
--insert sample data into students table
INSERT INTO students (student_id, name, age) VALUES
(1, 'Alice', 20),
(2, 'Bob', 22),
(3, 'Charlie', 19);
--insert sample data into subjects table
INSERT INTO subjects (subject_id, subject_name) VALUES
(1, 'Mathematics'),
(2, 'Physics'),
(3, 'Chemistry');
--insert sample data into exams table
INSERT INTO exams (exam_id, subject_id, exam_date) VALUES
(1, 1, '2024-07-01'),
(2, 2, '2024-07-02'),
(3, 3, '2024-07-03');
--insert sample data into results table
INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(1, 1, 1, 85.5),
(2, 1, 2, 90.0),
(3, 2, 1, 78.0),
(4, 2, 3, 92.5),
(5, 3, 2, 88.0);
```

```sql
-- Retrieve subject-wise marks for a student.
SELECT s.name AS student_name, sub.subject_name, r.score
FROM students s
JOIN results r ON s.student_id = r.student_id
JOIN exams e ON r.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
WHERE s.student_id = 1;
-- Calculate average marks for each subject
SELECT sub.subject_name, AVG(r.score) AS average_score
FROM subjects sub
JOIN exams e ON sub.subject_id = e.subject_id
JOIN results r ON e.exam_id = r.exam_id
GROUP BY sub.subject_name;
```

```sql
--fetch all students with their subjects and results
SELECT s.name AS student_name, sub.subject_name, r.score
FROM students s
JOIN results r ON s.student_id = r.student_id
JOIN exams e ON r.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id;
```

SQL ▾ 1 / 1 1 - 2 of 2

student_name	subject_name	score
Alice	Mathematics	85.5
Alice	Physics	90.0

SQL ▾ 1 / 1 1 - 3 of 3

subject_name	average_score
Chemistry	92.5
Mathematics	81.75
Physics	89.0

SQL ▾ 1 / 1 1 - 5 of 5

student_name	subject_name	score
Alice	Mathematics	85.5
Alice	Physics	90.0
Bob	Mathematics	78.0
Bob	Chemistry	92.5
Charlie	Physics	88.0